

Spécification et preuve de programmes

Solution du devoir 1 2025-26

Alain Giorgetti

Master d'Informatique de l'Université Marie et Louis Pasteur

1 Formaliser (5 points)

Soit A et B sont des variables propositionnelles, susceptibles de représenter n'importe quelle proposition. Formaliser, à l'aide de connecteurs logiques appropriés, les énoncés suivants :

1. « A si B »
2. « A est condition nécessaire pour B »
3. « A sauf si B »
4. « A seulement si B »
5. « A est condition suffisante pour B »
6. « A bien que B »
7. « Non seulement A , mais aussi B »
8. « A et pourtant B »
9. « A à moins que B »
10. « Ni A , ni B »

- | |
|--|
| <ol style="list-style-type: none">1. “ A si B ” : $B \Rightarrow A$.2. “ A est condition nécessaire pour B ” : $B \Rightarrow A$.3. “ A sauf si B ” : $(B \Rightarrow \neg A) \wedge (\neg B \Rightarrow A)$, autrement dit $A \Leftrightarrow \neg B$.4. “ A seulement si B ” : $A \Rightarrow B$.5. “ A est condition suffisante pour B ” : $A \Rightarrow B$.6. “ A bien que B ” : $A \wedge B$.7. “ Non seulement A, mais aussi B ” : $A \wedge B$.8. “ A et pourtant B ” : $A \wedge B$.9. “ A à moins que B ” signifie “ A sauf si B ” : $A \Leftrightarrow \neg B$.10. “ Ni A, ni B ” : $\neg A \wedge \neg B$. |
|--|

2 Preuve propositionnelle avec Why3 (2 points)

 Donner le contenu d'un fichier WhyML qui permet de déterminer si l'ensemble de formules propositionnelles

$$\{p \vee q, \neg q \vee r, p \vee \neg r, \neg p \vee q, \neg p \vee \neg q\}$$

est contradictoire ou non.

(1 point pour la syntaxe, 1 point si la formule correspond à l'énoncé)

Pour montrer qu'un ensemble de formules est contradictoire, on peut par exemple démontrer que la négation d'une d'entre elles est impliquée par la conjonction des autres formules. Une solution est dans le listing suivant :

```
predicate p
predicate q
predicate r
```

```
goal f1: (p ∨ q) ∧ (not q ∨ r) ∧ (p ∨ not r) ∧ (not p ∨ q) → not (not p ∨ not q)
```

Ce n'est pas demandé dans l'énoncé, mais on peut ajouter que l'interface en ligne de Why3 démontre automatiquement ce but.

3 Logique du premier ordre avec Why3 (5 points)

1. Expliquer clairement et simplement pourquoi les formules

$$(\forall z. s(a, z)) \Rightarrow \exists u. s(u, b)$$

et

$$(\forall z. s(a, z)) \Rightarrow s(a, b)$$

sont vraies quels que soient le prédicat s et les constantes a et b .

(1 point) En français, la seconde formule se lit : “Si $s(a, z)$ est vrai pour tout z , alors $s(a, b)$ est vrai.”. Cette formule se démontre simplement en choisissant b comme valeur de z dans l'hypothèse. Par conséquent, cette seconde formule est vraie quels que soient le prédicat s et les constantes a et b .

(1 point) La première formule se lit : “Si $s(a, z)$ est vrai pour tout z , alors il existe t tel que $s(t, b)$ est vrai.”. Cette formule se démontre simplement en choisissant a comme valeur de t . On obtient alors la seconde formule, qu'on a démontré ci-dessus. Par conséquent, cette première formule est aussi vraie quels que soient le prédicat s et les constantes a et b .

2.  Après avoir recopié les déclarations suivantes dans le fichier `devoir1spp.mlw`, formaliser ces deux formules en WhyML, sous la forme de deux buts.

```
type t
predicate s t t
constant a : t
constant b : t
```

(2 points)

```
goal g1 : (forall z:t. s a z) → exists u:t. s u b
goal g2 : (forall z:t. s a z) → s a b
```

3. Sans justifier, donnez les résultats de la vérification de ces deux buts dans l'interface en ligne de Why3.

(1 point) L'interface en ligne de Why3 démontre automatiquement ces deux buts.

4 Type et théorie de formules propositionnelles en WhyML (3 points)

1.  Définir un type WhyML correspondant au type $pf(ap)$ du cours sur les types inductifs, en remplaçant ap par une variable de type, par exemple α .

(1 point)

```
type pf 'a = Atom 'a | Non (pf 'a) | Ou (pf 'a) (pf 'a) | Et (pf 'a) (pf 'a)
          | Imp (pf 'a) (pf 'a) | Eqv (pf 'a) (pf 'a)
```

2.  Définir en WhyML des fonctions sur le type défini dans la question précédente, correspondant au quatre premières formules de l'exercice 2.11 du cours.

(2 points)

```
let function f1 (p q : pf 'a) = Imp (Et p q) p
let function f2 (p q : pf 'a) = Imp (Ou p q) (Et p q)
let function f3 (p q : pf 'a) = Imp (Et p q) (Ou p q)
let function f4 (p q : pf 'a) = Imp p (Ou p q)
```

Les définitions avec seulement le mot-clé `let` ou seulement le mot-clé `function`, ou avec $(p\ q : 'a)$ pour les paramètres des fonctions, sont aussi considérées comme correctes.

5 Autre type d'arbres binaires (2 points)

1. Définir des constructeurs pour un type d'arbres binaires stockant des données de type α non pas sur ses feuilles mais uniquement dans les nœuds internes.

(1 point) Le type inductif $itree(\alpha)$ des arbres binaires à nœuds internes de type α peut être défini par les deux constructeurs suivants :

$$Empty : arbre(\alpha) \tag{1}$$

$$Inode : \alpha \times arbre(\alpha) \times arbre(\alpha) \rightarrow arbre(\alpha) \tag{2}$$

2.  Donner le code WhyML qui permet de déclarer ce type.

(1 point)

```
type itree 'a = Empty | Inode 'a (itree 'a) (itree 'a)
```

6 Propriétés de tableaux d'entiers WhyML (3 points)

Le prédicat suivant spécifie en WhyML la propriété que tous les éléments du tableau d'entiers `a` ont la même valeur `c`.

```
use int.Int
use array.Array
```

```
predicate cte (a: array int) (c: int) = forall i. 0 ≤ i < length a → a[i] = c
```

Sur le même modèle, spécifier en WhyML les propriétés suivantes d'un tableau d'entiers quelconque, nommé `t`, en respectant les contraintes de l'énoncé.

1.  Avec un prédicat nommé `nextSame`, spécifier que chaque élément du tableau `t`, sauf le dernier, est égal à son élément suivant.

(1 point)

```
predicate nextSame (t : array int) = forall i. 1 ≤ i < length t → t[i] = t[i-1]
```

2.  Avec un prédicat nommé, spécifier que le début du tableau `t` est croissant, jusqu'à un certain indice `i` inclus, puis que la suite du tableau est décroissante. On pourra supposer que l'entier `i` est compris entre 0 et `length t-1` inclus.

(1 point)

```
predicate p2 (t : array int) (i: int) = (forall k. 0 ≤ k < i → t[k] ≤ t[k+1])
  ∧ forall k. i ≤ k < length t-1 → t[k] ≥ t[k+1]
```

3.  Avec un prédicat nommé, spécifier que le tableau `t` est de longueur paire et contient alternativement les entiers 0 et 1 : `t = { 0, 1, 0, 1, ..., 0, 1 }` (sans utiliser `mod`).

(1 point)

```
predicate p3 (t : array int) = exists k. length t = 2*k ∧
  forall i. 0 ≤ i < k → t[2*i] = 0 ∧ t[2*i+1] = 1
```